

EECS 482

Introduction to operating systems

Cooperating threads

Peter Chen
University of Michigan

Review

- **Process**: one or more threads in an address space
 - **Thread**: a sequence of executing instructions
 - + Simplicity: program can issue synchronous/blocking requests
 - + Performance: allow multiple slow things to happen in parallel
 - Two types of sharing
 - Application resource
 - ~~Hardware resource~~
- For now, assume each thread has its own processor (no sharing of processors)

Non-deterministic ordering → Non-deterministic results

- Assignment example (x is initially 0)

Thread A

x = 1

X = -100
X += 101

Thread B

x = -1

X = -100
X += 99

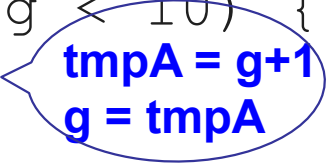
- Possible results: 1, -1
- Impossible results: 100, etc.?

Atomic operations

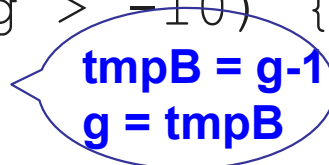
- Before we can reason at all about cooperating threads, we must know that some operation is **atomic**
 - Indivisible, i.e., happens in its entirety or not at all
 - No events from other threads can occur in between start and end of an atomic operation
- Thread 1: print ABC Thread 2: print 123
 - What if each print statement were atomic?
 - What if printing a single character were **not** atomic?
- Need an atomic operation to build a bigger atomic operation
- Set of atomic instructions varies by ISA
 - E.g., memory load is atomic; memory store is atomic; many other instructions are not atomic (e.g., double-precision floating point)

Example

Thread A

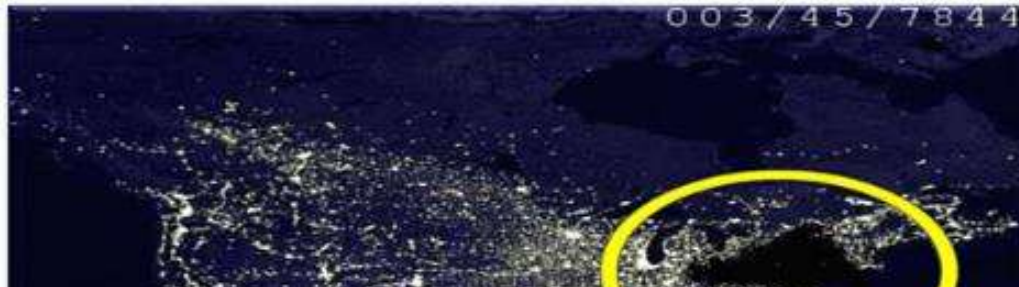
```
g=0
while (g < 10) {
    g++ 
}
print "A finished"
```

Thread B

```
g=0
while (g > -10) {
    g-- 
}
print "B finished"
```

- Which thread will finish first?
- Is the winner guaranteed to print first?
- Is it guaranteed that someone will win?
- Say both threads run at **exactly** the same speed, and start close together
 - Is it guaranteed that both threads will loop forever?

Do you really need to worry about such far-fetched scenarios?



ISAT GeoStar 45
23:15 EST 14 Aug. 2003

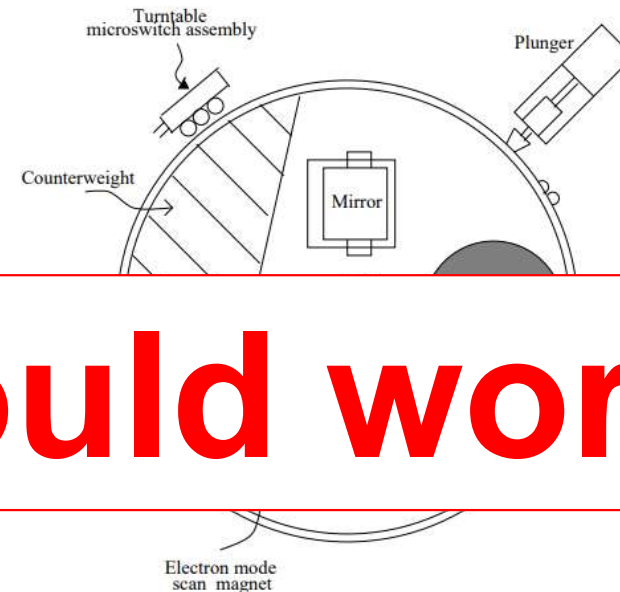


Figure 1: Upper turntable assembly.

Yes, you should worry!

Northeast blackout of 2003

Therac-25 radiation therapy
[Leveson95]

Non-deterministic interleaving makes debugging challenging

Heisenbug: a bug that occurs non-deterministically

Writing correct concurrent programs

- Consider and control all possible interleavings of events from multiple threads
 - All possible interleavings of atomic operations must result in correct output
 - Non-atomic operations to a shared resource must not occur concurrently
- Controlling how events from different threads can interleave is called **synchronization**

Synchronization

- Goals
 - **Eliminate interleavings**: all possible interleavings must produce a correct result
 - » Trivial solution?
 - **Preserve interleavings**: constrain thread interleavings as little as possible
- Concurrency only matters for events that access some shared resource
- A correct concurrent program should work no matter how fast the processors are that execute the threads

Administration

- Project 1 is posted on web page
 - Write a concurrent program
 - Material covered this week. For now:
 - » Read handout
 - » Compile and run the sample program
 - » Create threads
- Pre-lab questions for Friday's lab section

Too much milk

- Problem definition
 - Janet and Peter want to keep their refrigerator stocked with at most one milk jug
 - If either sees fridge empty, she/he goes to buy milk

Solution #0: No synchronization

Peter

```
if (no milk) {  
    buy milk  
}
```

Janet

```
if (no milk) {  
    buy milk  
}
```

Problem?

First type of synchronization: mutual exclusion

- Mutual exclusion
 - Ensure that only 1 thread is doing a certain thing at one time (other threads are excluded). E.g., only 1 person goes shopping at a time.
 - Constrains interleavings of threads: “not at the same time”
 - Does this remind you of any other concept we’ve talked about?

Critical section

- A section of code that needs to be run atomically with respect to selected other pieces of code (because they access some shared resource)
- If code A and code B are critical sections with respect to each other, then multiple threads should not be able to interleave events from A and B
- In Too much milk, critical section is:

```
if (no milk) {  
    buy milk  
}
```
- Mutual exclusion: how to make a critical section atomic?

Solution #1

- Assume the only atomic operations are **leave note**, **remove note**, and **check note**
- Leave note that you're going to check on the milk, so other person doesn't also buy

Peter

```
if (no note) {  
    leave note  
    if (no milk) {  
        buy milk  
    }  
    remove note  
}
```

Janet

```
if (no note) {  
    leave note  
    if (no milk) {  
        buy milk  
    }  
    remove note  
}
```

- Does this work?
- Is solution #1 better than solution #0?

Too much milk (solution #2)

- Change the order of “leave note” and “check note”. Notes need to be labelled (otherwise you’ll see your note and think the other person left it).

Peter

```
leave notePeter
if (no noteJanet) {
    if (no milk) {
        buy milk
    }
}
remove notePeter
```

Janet

```
leave noteJanet
if (no notePeter) {
    if (no milk) {
        buy milk
    }
}
remove noteJanet
```

- Does this work?

Too much milk (solution #3)

- Decide who will buy milk when both leave notes at the same time. Peter hangs around to make sure job is done.

Peter

leave notePeter

while (noteJanet) {
do nothing

}

if (no milk) {
buy milk

}

remove notePeter

Janet

leave noteJanet

if (no notePeter) {

if (no milk) {
buy milk

}

}

remove noteJanet

- Peter's "while (noteJanet)" prevents him from entering the critical section at the same time as Janet

Proof of correctness

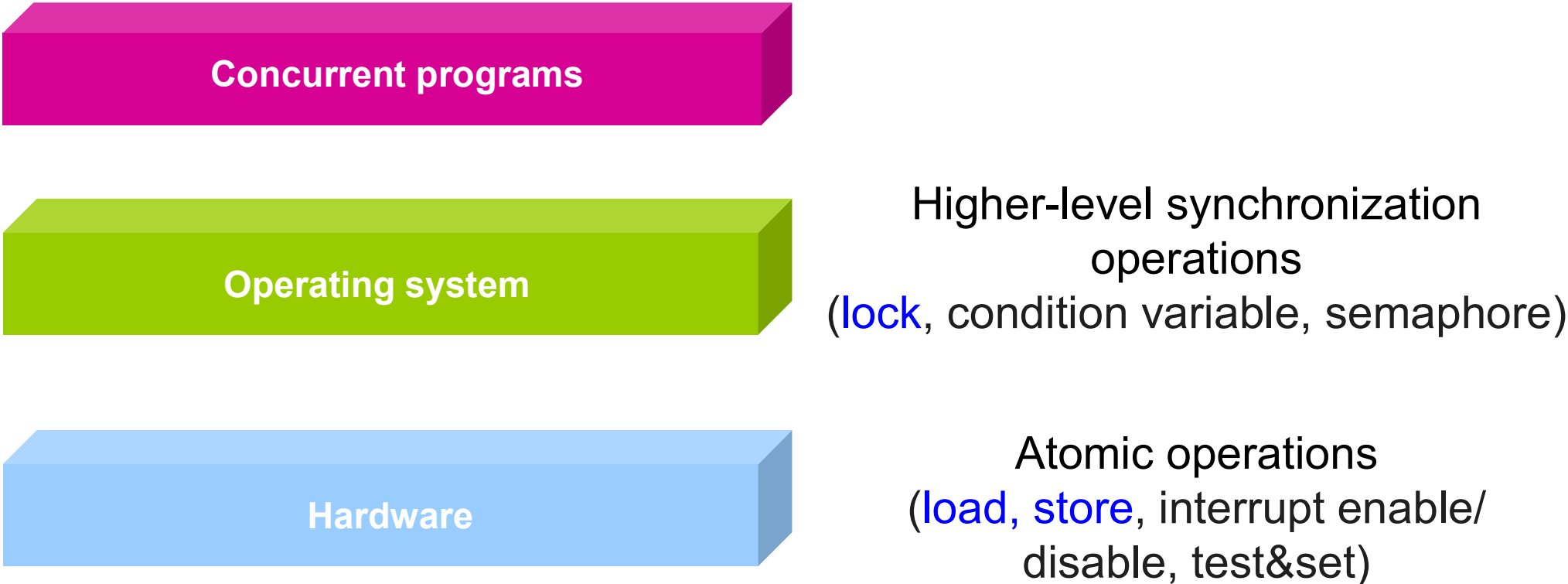
- Janet
 - if no notePeter, then Peter hasn't started yet, so it's safe to buy. Peter will wait for Janet to be done before checking.
 - if notePeter, then Peter is in the body of the code and will eventually buy milk if needed. Note that Peter may be waiting for Janet to exit.
- Peter
 - if no noteJanet, it's safe to buy. Peter has already left notePeter, and Janet will check notePeter in the future.
 - if noteJanet, Peter waits to see what Janet does
 - » Janet may have checked notePeter before Peter left note. In this case, Janet will buy milk.
 - » Janet may have checked notePeter after Peter left note. In this case, Janet will not buy milk.

Analysis of solution #3

- Pros
 - + It works!
 - + Works even if most operations are not atomic
- Cons
 - Complicated; not obviously correct
 - Asymmetric
 - Not obvious how to scale to more people
 - Peter consumes CPU time while waiting. This is called **busy-waiting**.

Higher-level synchronization

- Raise the level of abstraction to make life easier for programmers



Concurrent programs

Operating system

Higher-level synchronization
operations
(**lock**, condition variable, semaphore)

Hardware

Atomic operations
(**load**, **store**, interrupt enable/
disable, test&set)

Locks (mutexes)

- A lock prevents another thread from entering a critical section
 - *“Lock fridge while checking milk status and shopping”*
- Two operations
 - **lock()**: wait until lock is free, then acquire it

```
while (1) {  
    if (lock is free) {  
        acquire lock  
        break  
    }  
}
```

**atomic**
 - **unlock()**: **atomic**
- **Problems?** Hint: why was the note in *Too much milk* (solutions #1 and #2) not a good lock?

Locks (mutexes)

- How to use locks
 - Lock is initialized to free
 - Thread should acquire lock before entering critical section
 - Thread that acquired lock should release lock when done with critical section

```
lock()  
<critical section>  
unlock()
```

- All synchronization involves waiting
- Thread can be running or blocked

Locks make “Too much milk” easy!

mutex milk;

Peter

```
milk.lock()  
if (noMilk) {  
    buy milk  
}  
milk.unlock()
```

Janet

```
milk.lock()  
if (noMilk) {  
    buy milk  
}  
milk.unlock()
```

Efficiency

- But this prevents Janet from doing things while Peter is buying milk

Peter

milk.lock()

if (no milk) {
 buy milk

}

milk.unlock()

Janet

milk.lock()

if (no milk) {
 buy milk

}

milk.unlock()

- How to minimize the time the lock is held?

Efficiency – remember Solution #1?

```
note.lock()
if (no note) {

    leave note
    note.unlock()

    if (no milk) {
        buy milk
    }

    note.lock()
    remove note

}
note.unlock()
```